

AXIOM RedAgent (adversarial probe)

Eight-vector attack agent that probes the guard stack (`/guard/check` by default) with deliberately hostile payloads, records each response with latency, and emits a single HMAC-signed `AttackReport` .

The implementation lives in `axiom_red_agent.py:1` . Trust constants are frozen at module load — assigning to `TRUST_LEVEL` or `ISOLATION` raises `AttributeError` (see `_FrozenModule` at `axiom_red_agent.py:51`).

Trust constraints

Field	Value	CANNOT_MUTATE
<code>TRUST_LEVEL</code>	1	yes
<code>ISOLATION</code>	True	yes

The agent only writes to the guard endpoint — it cannot touch production. The guard URL defaults to `http://localhost:8001/guard/check` and can be overridden in the constructor.

Attack vectors

#	Vector	Intent
1	<code>PROMPT_INJECTION</code>	Override system instructions
2	<code>TRUST_ESCALATION</code>	Raise trust level via fake authority
3	<code>ISOLATION_BYPASS</code>	Disable sandbox flags
4	<code>CANNOT_MUTATE_FIELD</code>	Reassign frozen constitutional constants
5	<code>GUARD_DISABLE</code>	Turn off active guards
6	<code>SUPPLY_CHAIN_TAMPER</code>	Submit forged HMAC for a manifest update
7	<code>ENCODING_CONFUSION</code>	Cyrillic / diacritic homoglyphs (BUG-008)
8	<code>PREFLIGHT_SKIP</code>	Bypass preflight validation

Each vector exists as `attack_<name>()` on `RedAgent` — call individually or use `run_all_attacks()` for the signed aggregate.

Hard limits

- `MAX_RESPONSE_BYTES = 65_536` — responses larger than this are recorded as `response_too_large` and not parsed
- `_REQUEST_TIMEOUT_S = 5.0` — per-request HTTP timeout
- BUG-010 mitigation: `_parse_response` checks `len(resp.content) > 0` before any index access

CLI

```
python axiom_red_agent.py
```

Runs all 8 vectors against the default guard URL and prints a coloured BLOCKED / BYPASSED table with per-vector latency and the signed report HMAC header.

API

```
from axiom_signing import derive_key
from axiom_red_agent import RedAgent

key = derive_key(b"axiom-red-agent-v1")
agent = RedAgent(hmac_key=key,
                 guard_url="http://localhost:8001/guard/check")
report = agent.run_all_attacks()
# report.results      - list of AttackResult
# report.signature    - hex HMAC over the report body
# report.timestamp    - UTC ISO-8601
```

Pairing with BlueAgent

The standard workflow is RedAgent → BlueAgent: pass the `AttackReport` into `BlueAgent.run_all_defenses(...)` for a matched detection pass. See `BLUE_AGENT.md`.

Encoding hygiene

The agent enforces UTF-8 across stdout, stderr, HTTP body, and HMAC payloads (BUG-003 / BUG-008). All payload strings are `.encode("utf-8")`-ed before HMAC and the digest is always finalised via `.hexdigest()` (BUG-007).